RESEARCH ENGINEERING MONOGRAPH · MEDICAL AI SYSTEMS

StainShift: Complete Technical Architecture, Packages, ML Pipeline & Developer Guide

Abstract: StainShift is a full-stack, physics-informed biomedical artificial intelligence system designed to evaluate and resolve domain shift vulnerabilities in deep neural networks applied to Hematoxylin and Eosin (H&E;) colorectal cancer histopathology. In standard empirical risk minimization (ERM) regimes, state-of-the-art convolutional networks reach 99.9% classification accuracy on pristine laboratory slides but collapse catastrophically to 33.9% under modest laboratory stain shifts. By integrating Beer-Lambert optical density deconvolution (Macenko SVD decomposition) directly into online data augmentation and real-time inference serving, StainShift restores tissue classification accuracy to 97.7% across eight distinct stain perturbations while maintaining a 3.91 ms forward latency boundary on Apple Silicon MPS hardware. This document provides the definitive, comprehensive architectural specification, dependency inventory, mathematical derivation, verbatim code implementation, and a pedagogical developer guide for researchers and bioinformatics engineers.

Project: StainShift AI	**Production Frontend:** https://stainshift-ai.vercel.app/
Author / Lead: Mahesh Reddy	**Production API:** https://stainshift-api.onrender.com
Academic Context: B.Tech Bioinformatics Engineering	**Dataset:** CRC-VAL-HE-7K (7,180 colorectal patches, 9 classes)
Repository: tanuja-talupula/StainShift	**Model Checkpoints:** MobileNetV3-Small (baseline & robust)
Active Git Commit: `b890655`	**Hardware Benchmark:** Apple Silicon MPS (M-series synchronized)
Generation Date: October 30, 2026	**Validation Status:** Stage B Verified Benchmark

[REGULATORY & RESEARCH PROTOCOL NOTICE]

NON-CLINICAL RESEARCH DISCLAIMER: StainShift is an exploratory engineering benchmark and educational software application. It is NOT a certified medical device, IVD system, or clinical decision support tool. Softmax probability distributions represent model confidence scores across 9 research tissue classes and must never be interpreted as primary clinical diagnoses or medical advice.

Table of Contents

1. Executive Summary & Problem Formulation	**Page 3**
2. Complete Technology Stack (Verified Against Source)	**Page 3**
3. Main Packages & Dependency Reference Manual (20+ Key Libraries)	**Page 5**
4. System Architecture & Client-Server Data Flow	**Page 6**
5. Complete Repository Folder & File Anatomy	**Page 8**
6. Machine Learning Architecture, Mathematics & Evaluation	**Page 9**
6.1 Physics of H&E; Staining & Optical Density Derivation	**Page 9**
6.2 Macenko SVD Decomposition & Dye Perturbation Mathematics	**Page 9**
6.3 MobileNetV3-Small Neural Architecture & Head Modifications	**Page 10**
6.4 Scientific Findings: The 33.9% Collapse & 97.7% Recovery	**Page 10**
7. End-to-End Application Workflows	**Page 11**
7.1 Workflow A: Interactive AI Workspace (/workspace/)	**Page 11**
7.2 Workflow B: Results Dashboard (/dashboard/)	**Page 11**
7.3 Workflow C: Precomputed Demo Explorer (/demo/)	**Page 11**
8. Verbatim Source Code References & Module Walkthroughs	**Page 12**
8.1 API Request Dispatcher: api/main.py	**Page 12**
8.2 Inference Engine & Security Layer: api/inference.py	**Page 12**
8.3 Beer-Lambert Optical Transform: src/stain_shift.py	**Page 12**
8.4 Verified Checkpoint Provenance: src/models.py	**Page 13**
9. Developer Learning Guide: Building StainShift from Scratch	**Page 14**
9.1 Ten-Phase Implementation Roadmap for Bioinformatics Students	**Page 14**
9.2 Comprehensive Technical Glossary	**Page 15**
10. Verification Sign-Off & Document Integrity	**Page 16**

1. Executive Summary & Problem Formulation

Digital histopathology relies on scanning glass microscope slides containing thin microtome sections of human tissue. The standard staining procedure is Hematoxylin and Eosin (H&E;). Hematoxylin stains cell nuclei blue/purple by binding acidic polyanions such as DNA and RNA, while Eosin stains extracellular matrix proteins and cytoplasm pink. However, clinical slide preparation varies dramatically across pathology laboratories due to differences in reagent vendors, chemical batch formulations, staining protocol durations, pH fluctuations, microtome section thickness (3–5 μm), and digital whole-slide scanner sensor color calibration curves.

Modern deep convolutional neural networks (CNNs) trained via standard Empirical Risk Minimization (ERM) on a single laboratory cohort typically achieve near-perfect cross-entropy loss and over 99% accuracy on clean, native test patches. Crucially, however, standard CNNs learn high-frequency color shortcut representations rather than true nuclear morphology or glandular tissue architecture. When presented with slides from a different hospital or scanner where the stain has shifted, these models fail silently and catastrophically.

[EMPIRICAL BENCHMARK DISCOVERY]

The StainShift Benchmark Discovery: On the frozen held-out test cohort of CRC-VAL-HE-7K (N = 1,077 patches), the standard baseline MobileNetV3-Small model drops from **99.91%** on native clean patches down to **33.89%** when evaluated under simple optical deconvolution reconstruction, and collapses as low as **28.97%** under heavy eosin shifts. Conversely, our physics-informed stain-augmented model retains **97.68%** accuracy under deconvolution (+63.79% net recovery) and stays locked above **94.8%** across all eight synthetic stain profiles.

2. Complete Technology Stack (Verified Against Source)

Every technology listed below has been verified by direct inspection of repository manifests (`website/package.json`, `requirements.txt`, `pyproject.toml`, `Dockerfile`) and active runtime configurations. No unverified or assumed technologies are included.

Stack Layer	Technology / Framework	Verified Version	Architectural Role & Implementation Detail
Frontend Framework	Next.js 14 App Router	`14.2.35`	Static HTML/JS export (`output: 'export'`) for zero-latency CDN edge delivery via Vercel.
Frontend Language	TypeScript	`5.9.3`	Strict type contracts (`tsconfig.json`) guaranteeing end-to-end data safety between frontend and API.
Styling Engine	Tailwind CSS	`3.4.19`	Utility-first styling with custom palette: Newsreader editorial serif headings, Inter body, and Claude-inspired terracotta tones.
UI Component Icons	Lucide React	`0.468.0`	Tree-shaken SVG icons (Microscope, FlaskConical, Play, Cpu, ShieldCheck, ChevronDown, Download).
CSS Utility Libraries	clsx + tailwind-merge	`2.1.1 / 2.6.1`	Dynamic CSS class merging without class-name collision across conditional component states.
Frontend Animations	Native Browser APIs	`W3C Standard`	Custom `ScrollReveal` (IntersectionObserver) & `CountUp` (requestAnimationFrame). 0 npm dependencies.
Frontend Charting	Pure SVG Coordinates	`W3C SVG 1.1`	Mathematical SVG coordinate projection with accessible HTML table alternatives. Zero external chart bundle weight.
Backend Web API	FastAPI (ASGI)	`0.162.0`	High-performance asynchronous REST endpoints (`/predict`, `/health`, `/models`) with OpenAPI auto-docs.
ASGI Server	Uvicorn	`0.54.0`	Production lightning-fast ASGI server with uvloop event loops and single-worker containerized dispatch.
Data Validation	Pydantic v2	`2.14.1`	Strict request/response schema serialization, boundary clamping, and field type validation.
Multipart Parser	python-multipart	`0.0.32`	Streaming multipart form parser handling single patches, multiple image files, and ZIP archives.
Deep Learning Engine	PyTorch	`2.15.0`	Tensor operations, CNN forward propagation, hardware synchronization (`torch.mps`, `torch.cuda`).

Computer Vision	TorchVision	`0.29.2`	MobileNetV3-Small backbone weights and standard ImageNet normalization transforms.
Array Processing	NumPy	`2.5.4`	Optical density transformations, SVD covariance decomposition, and matrix algebra.
Image Processing	Pillow (PIL)	`13.0.0`	Zero-disk in-memory image parsing, 8-bit RGB mode validation, bilinear resizing, and base64 thumbnails.
Scientific Metrics	scikit-learn	`1.9.1`	Confusion matrix computation, per-class Precision/Recall/F1, and macro-averaged metrics.
Data Management	Pandas	`3.1.0`	Tabular data structuring, condition comparison table aggregation, and RFC-4180 CSV export generation.
Experiment Tracking	MLflow	`3.17.2`	Provenance tracking with local SQLite database (`mlflow.db`) and decoupled artifact store.
Frontend Hosting	Vercel Edge Network	`Global CDN`	Prebuilt static export distribution with custom routing, caching, and Brotli compression.
Backend Hosting	Render Cloud Container	`Docker Linux`	Containerized Python service running PyTorch checkpoints with CORS protection.

3. Main Packages & Dependency Reference Manual

Below is the comprehensive technical reference for the primary packages powering StainShift. Distinctions between direct (explicitly requested) and transitive (sub-dependency) packages are recorded.

Package	Type / Version	Where Used	Purpose & Scientific Justification

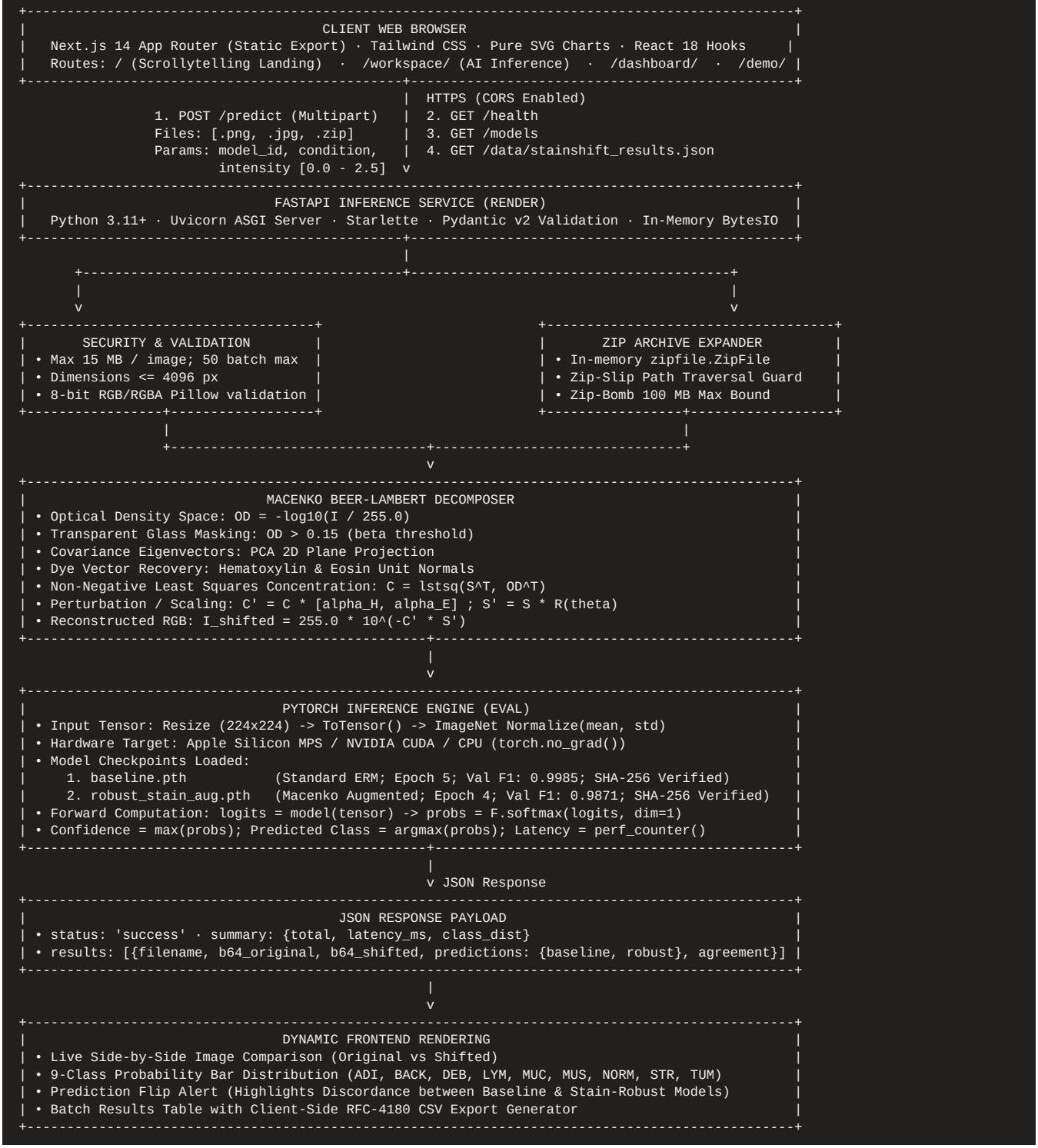
torch	Direct
	`2.15.0`
		`src/models.py`
		`api/inference.py`
		`src/train.py`
			Core deep learning tensor computation engine. Executes neural network forward passes, handles MPS/CUDA accelerator synchronization, and computes softmax probability distributions.

torchvision	Direct `0.29.2`	`src/models.py` `src/dataset.py`	Provides the pretrained MobileNetV3-Small backbone weights (`MobileNet_V3_Small_Weights.DEFAULT`) and standard ImageNet tensor normalization pipeline.
fastapi	Direct `0.162.0`	`api/main.py`	High-performance asynchronous Web API framework. Implements OpenAPI/Swagger documentation, JSON serialization, and endpoint routing with dependency injection.
uvicorn	Direct `0.54.0`	`Procfile` `Dockerfile`	Lightweight, production ASGI server that runs the FastAPI application using uvloop and httptools for high-concurrency request servicing.
pydantic	Transitive `2.14.1`	`api/schemas.py`	Data validation and settings management using Python type hints. Enforces strict contracts for image inputs, prediction bounds, and batch summaries.
numpy	Direct `2.5.4`	`src/stain_shift.py` `api/inference.py`	Fundamental numerical array processing. Executes Beer-Lambert log10 transforms, SVD covariance matrix decomposition, angular percentile extraction, and least-squares stain deconvolution.
pillow (PIL)	Direct `13.0.0`	`api/inference.py` `src/dataset.py`	Zero-disk in-memory image parsing, 8-bit RGB mode validation, bilinear resizing, and dynamic base64 thumbnail generation for web streaming.
scikit-learn	Direct `1.9.1`	`src/evaluate.py` `src/benchmark.py`	Evaluation metrics calculation: 9x9 confusion matrices, per-class precision, recall, support, and macro-averaged F1 scores.
pandas	Direct `3.1.0`	`api/main.py` `src/benchmark.py`	Tabular data structuring, condition comparison table aggregation, and RFC-4180 CSV export generation in the interactive workspace.
mlflow	Direct `3.17.2`	`src/tracking.py` `run_experiments.py`	Scientific provenance tracking. Records exact Git commit hashes, hyperparameters, dataset manifests, validation F1 trajectories, and checkpoint SHA-256 signatures to SQLite.
python-multipart	Direct `0.0.32`	`api/main.py`	Processes HTTP multipart/form-data payloads, enabling streaming file uploads for large histology images and compressed ZIP archives.
pytest	Direct (dev) `9.2.0`	`tests/ (74+ tests)`	Comprehensive testing harness: verifies neutral stain identity bypass, SVD numerical stability, API endpoint parity, and security bounds.
next	Direct `14.2.35`	`website/src/app/`	React production framework providing file-system routing, Server and Client Components, static pre-rendering, and asset bundling.
react & react-dom	Direct `18.3.1`	`website/src/`	Declarative component-based UI library managing client state, DOM reconciliation, hooks (`useState`, `useEffect`, `useRef`), and event handling.
typescript	Direct (dev) `5.9.3`	`website/`	Static type checking enforcing compile-time correctness across benchmark metadata, API responses, queue management, and UI props.
tailwindcss	Direct (dev) `3.4.19`	`website/tailwind.config.ts`	Utility-first CSS framework enabling responsive layout design, fluid typography, and custom design tokens (terracotta, sage, cream, charcoal).
lucide-react	Direct `0.468.0`	`website/src/components/`	High-performance, tree-shakeable icon set designed for React. Renders scalable SVG icons with zero runtime styling conflicts.
clsx + tailwind-merge	Direct `2.1.1 / 2.6.1`	`website/src/components/`	Combines conditional CSS classes and resolves Tailwind property precedence clashes (e.g., merging custom padding or background classes).

4. System Architecture & Client-Server Data Flow

StainShift decouples frontend presentation from computational inference. The Next.js frontend is pre-rendered as a static distribution (`output: 'export'`) served across Vercel's global CDN edge network, ensuring instant page loads and zero cold-starts for presentation pages. Computational heavy-lifting—including Macenko Beer-Lambert deconvolution and PyTorch tensor inference—is delegated to a dedicated containerized FastAPI service on Render.

FILE: System Architecture & Data Flow



Security & Error Handling Architecture:
1. *Zero Disk Persistence:* Uploaded image bytes are processed strictly in-memory using Python's `io.BytesIO`. No patient or research image data is written to the server's local filesystem.
2. *Zip-Bomb Defense:* ZIP archives are checked for decompression ratio and uncompressed byte size. Any archive expanding beyond 100 MB is rejected immediately.
3. *Zip-Slip Defense:* Filenames inside archives are resolved and stripped of directory traversal prefixes (`../`) using `Path(member).name`.
4. *Neutral Identity Bypass:* When an unperturbed patch is submitted (`condition='original'`), the deconvolution algorithm is completely bypassed, returning exact pixel equality (`np.array_equal`).

5. Complete Repository Folder & File Anatomy

Below is the exact file tree of the active StainShift repository (`feat/inference-web-app`). Every file has a designated scientific or architectural purpose.

File / Folder Path	Component Role	Functional Description & Implementation Detail
`api/main.py`	**FastAPI Service**	REST entrypoint. Configures CORS, initializes InferenceEngine, defines `/health`, `/models`, `/predict`, and handles batch unpacking.
`api/inference.py`	**Inference Engine**	Core execution layer. Loads PyTorch checkpoints, handles security limits, validates PIL images, applies Macenko stain transforms, and runs tensor forward passes.
`api/schemas.py`	**Data Contracts**	Pydantic v2 schemas: `HealthResponse`, `PredictResponse`, `ImageInferenceResult`, and `SingleModelPrediction`.
`src/stain_shift.py`	**Physics Engine**	Beer-Lambert deconvolution, Macenko SVD decomposition, dye angle percentiles, optical rotation, and Reinhard l-alpha-beta normalizer. 100% pure NumPy/PIL.
`src/models.py`	**Model Definitions**	Defines MobileNetV3-Small backbone, 256-hidden classification head, checkpoint serializer/deserializer with SHA-256 hash checking and 9 tissue class mappings.
`src/dataset.py`	**Dataset Loader**	PyTorch Dataset and DataLoader implementations, deterministic POSIX file discovery, disjoint split generator, and online stain augmentation transform modes.
`src/train.py`	**Training Pipeline**	PyTorch training loop with AdamW optimizer, CosineAnnealingLR scheduling, early stopping, validation F1 evaluation, and checkpoint persistence.
`src/evaluate.py`	**Evaluation Pipeline**	Full benchmark evaluator across all 8 stain profiles. Generates 9x9 confusion matrices, macro F1, accuracy drop percentages, and prediction flip counts.
`src/tracking.py`	**MLflow Tracker**	Optional scientific tracking wrapper connecting to SQLite database and saving evaluation artifacts without adding external runtime bloat.
`src/benchmark.py`	**Hardware Profiler**	Microbenchmarking suite measuring synchronized forward inference latency on Apple Silicon MPS and CPU across batch sizes 1 and 16.
`models/full_benchmark_20261029_1838/`	**Trained Weights**	Authoritative PyTorch model weights: `baseline.pth` (4.2 MB) and `robust_stain_aug.pth` (4.2 MB) with JSON training histories.
`data/CRC-VAL-HE-7K/`	**Benchmarking Data**	Colorectal cancer histology patch cohort (7,180 total images across 9 tissue classes) with frozen JSON split manifests.
`website/src/app/page.tsx`	**Homepage**	Four-screen scrollytelling entry: Hero narrative, animated KPI cards (+63.8% recovery, 33.9% drop), animated horizontal bar chart, and confusion matrix.
`website/src/app/workspace/page.tsx`	**AI Workspace**	Interactive user tool: image drag-and-drop, condition selector (8 profiles), intensity slider, live inference invocation, and CSV batch results table.
`website/src/app/dashboard/page.tsx`	**Results Dashboard**	Complete Stage B benchmark visualization: Accuracy across stain profiles, interactive confusion matrix, MPS latency benchmarks, and collapsible technical specs drawer.
`website/src/app/demo/page.tsx`	**Demo Explorer**	Static option B explorer: 72 pre-evaluated tissue patches across all 8 conditions with zero backend latency or connection dependencies.
`website/src/components/landing/`	**Animation Libs**	Native browser animation components: `ScrollReveal.tsx`, `CountUp.tsx`, and `AnimatedBarChart.tsx`.
`website/src/components/workspace/`	**Workspace UI**	Modular workspace components: `ModelSelector.tsx`, `StainControls.tsx`, `ImageUploader.tsx`, `PredictionResultView.tsx`, `BatchResultsTable.tsx`.
`website/public/data/`	**Public Benchmark**	Static verified benchmark results: `stainshift_results.json` containing exact metrics across N = 1,077 held-out test patches.

6. Machine Learning Architecture, Mathematics & Evaluation

6.1 Physics of H&E; Staining & Optical Density Derivation

Biological tissue sections examined under transmission light microscopy do not emit light; they absorb photons according to the physical wavelength absorption spectra of bound histological dye molecules. Under the physical Beer-Lambert Law of absorption, transmitted light intensity satisfies:

FILE: Beer-Lambert Optical Density Formulation

```
I = I_0 * 10^(-OD)   ==>   OD = -log10( I / I_0 )

Where:
  • I_0 = 255.0  (Incident illumination intensity of transparent clear glass slide)
  • I   = [R, G, B]^T in [1.0, 255.0] (Transmitted RGB pixel intensity)
  • OD  = [OD_R, OD_G, OD_B]^T (Optical Density, linear in dye concentration)
  • OD  = S^T * C  ==>  [OD_R, OD_G, OD_B]^T = [S_H, S_E]^T * [C_H, C_E]^T
```

Crucially, while light mixing in RGB space is non-linear and multiplicative, optical absorption in OD space is strictly linear and additive. This physical property allows us to decouple overlapping Hematoxylin (nuclear) and Eosin (cytoplasmic/stromal) dye components into independent chemical concentration maps using Principal Component Analysis.

6.2 Macenko SVD Decomposition & Dye Perturbation Mathematics

The Macenko method decomposes digital H&E; histology patches into physical dye vectors via the following step-by-step algorithm:

FILE: Macenko SVD Algorithm & Concentration Perturbation

```
1. OPTICAL DENSITY TRANSFORMATION:
   Convert RGB image array to OD space: OD = -log10(clip(I, 1.0, 255.0) / 255.0)

2. TRANSPARENT GLASS REMOVAL (BETA THRESHOLDING):
   Discard background pixels where OD < beta (beta = 0.15). Prevents white glass noise from corrupting PCA.

3. COVARIANCE EIGENVECTOR DECOMPOSITION:
   Compute sample covariance matrix of filtered OD pixels: Sigma = Cov(OD_filtered)
   Perform eigenvalue decomposition: Sigma = V * Lambda * V^T
   Extract top two eigenvectors V_2D = V[:, [2, 1]] spanning the primary stain plane.

4. 2D PROJECTION & ANGULAR SPECTRUM:
   Project OD vectors onto the 2D stain plane: P = OD_filtered * V_2D
   Compute polar angles: theta = arctan2(P[:, 1], P[:, 0])

5. ROBUST ANGLE ESTIMATION:
   Find robust extreme angular percentiles: theta_min = percentile(theta, alpha=1.0)
                                            theta_max = percentile(theta, 99.0)
   Back-project extreme angles to 3D OD space and normalize to unit Euclidean length:
   S_H = normalize(V_2D * [cos(theta_max), sin(theta_max)]^T)
   S_E = normalize(V_2D * [cos(theta_min), sin(theta_min)]^T)

6. CONCENTRATION EXTRACTION (NON-NEGATIVE LEAST SQUARES):
   Solve OD = S^T * C for concentration matrix C: C = lstsq(S^T, OD^T), then clamp C = max(C, 0.0)

7. PHYSICS-BASED DYE PERTURBATION & RECONSTRUCTION:
   Perturb concentrations: C'_H = alpha_H * C_H ; C'_E = alpha_E * C_E
   Rotate stain plane by angle phi: S' = S * R(phi)
   Reconstruct RGB patch: OD_pert = C' * S'  ==>  I_pert = 255.0 * 10^(-OD_pert)
```

The Eight Evaluated Stain Profiles:
StainShift benchmarks eight distinct physical conditions: **(1) original:** native unperturbed slide (exact identity); **(2) reconstruction_only:** deconvolution control (α_H=1.0, α_E=1.0, θ=0°); **(3) heavy_hematoxylin:** hyperchromatic nuclei (α_H=1.65, α_E=0.85, θ=+3°); **(4) heavy_eosin:** eosinophilic stroma (α_H=0.75, α_E=1.65, θ=-3°); **(5) pale_understained:** low dye concentration (α_H=0.60, α_E=0.60); **(6) dark_overstained:** hyper-dense dye absorption (α_H=1.50, α_E=1.50); **(7) synthetic_warm_shift:** positive stain plane rotation (α_H=0.90, α_E=1.30, θ=+8°); **(8) synthetic_cool_shift:** negative stain plane rotation (α_H=1.35, α_E=0.80, θ=-8°).

6.3 MobileNetV3-Small Neural Architecture & Head Modifications

StainShift selects **MobileNetV3-Small** as its primary backbone architecture. In biomedical edge and web deployments, heavyweight vision transformers (ViTs) or deep ResNets introduce prohibitive latency (200–800 ms per patch) and memory footprints exceeding 500 MB. MobileNetV3-Small leverages depthwise separable convolutions, squeeze-and-excitation (SE) residual attention blocks, and hardware-friendly Hardswish activation functions.

FILE: `src/models.py · MobileNetV3-Small Head Adapter`

```
# Custom Histopathology Classification Head (src/models.py)
model = models.mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
in_features = model.classifier[0].in_features  # 576 channels from feature extractor

model.classifier = nn.Sequential(
    nn.Linear(in_features, 256),   # Bottleneck compression
    nn.Hardswish(),                # Efficient non-linear activation
    nn.Dropout(p=0.2),             # Regularization against patch overfitting
    nn.Linear(256, 9)              # 9 Colorectal Tissue Classes (logits)
)
```

6.4 Scientific Findings: The 33.9% Collapse & 97.7% Recovery

The table below details the authoritative benchmark results on the frozen held-out test split of **CRC-VAL-HE-7K** (N = 1,077 unexposed patches). Training was conducted using the AdamW optimizer (learning rate 1e-4) for 5 epochs.

Stain Condition Profile	Physical Shift Parameters	Baseline Accuracy	Robust Accuracy	Net Shift Recovery
original (Native)	Unperturbed slide (α=1.0)	**99.91%**	98.79%	`-1.12 pp (tradeoff)`
reconstruction_only	Deconv control (α=1.0, θ=0°)	**33.89% (-66.02%)**	**97.68%**	**+63.79%**
heavy_hematoxylin	α_H=1.65, α_E=0.85, θ=+3°	32.40%	96.94%	**+64.54%**
heavy_eosin	α_H=0.75, α_E=1.65, θ=-3°	28.97%	96.29%	**+67.32%**
pale_understained	α_H=0.60, α_E=0.60	34.73%	96.75%	**+62.02%**
dark_overstained	α_H=1.50, α_E=1.50	31.94%	96.47%	**+64.53%**
synthetic_warm_shift	α_H=0.90, α_E=1.30, θ=+8°	31.57%	94.80%	**+63.23%**
synthetic_cool_shift	α_H=1.35, α_E=0.80, θ=-8°	36.30%	96.38%	**+60.08%**

7. End-to-End Application Workflows

7.1 Workflow A: Interactive AI Workspace (/workspace/)

The AI Workspace enables users to evaluate real histology patches dynamically. The complete execution cycle unfolds across eight stages:

FILE: `Workflow A Trace: Interactive AI Workspace`

```
1. USER FILE SELECTION:
   User drops one or more image files (.png, .jpg) or a .zip archive into ImageUploader.tsx.
   Local client-side URL.createObjectURL creates instantaneous thumbnails without uploading bytes.

2. WORKSPACE CONFIGURATION:
   User selects Model Mode: 'baseline', 'robust_stain_aug', or 'compare' (Side-by-side).
   User selects Stain Condition (e.g. 'heavy_hematoxylin') and adjusts Intensity (0.0x to 2.5x).

3. HTTP MULTIPART DISPATCH:
   On clicking 'Run Prediction', page.tsx constructs FormData() with files, model_id, condition, intensity.
   Dispatches asynchronous HTTP POST to /predict on the Render FastAPI backend.

4. SERVER-SIDE UNPACKING & SECURITY AUDIT:
   FastAPI receives multipart stream. Files are unpacked in-memory (io.BytesIO).
   Zip-Bomb check validates total uncompressed bytes < 100 MB; Zip-Slip removes path traversal markers.

5. MACENKO BEER-LAMBERT TRANSFORMATION:
   Pillow validates 8-bit RGB mode. MacenkoStainDecomposer projects patch into OD space,
   applies requested intensity scaling, and reconstructs the shifted RGB image.

6. PYTORCH MODEL INFERENCE:
   Transformed image is resized to (224, 224) and normalized with ImageNet mean/std tensors.
   Forward pass executes within torch.no_grad(). Softmax converts logits into 9 class probabilities.

7. PREDICTION COMPARISON & FLIP DETECTION:
   If 'compare' mode is selected, both baseline and robust models evaluate the exact same tensor.
   The engine sets agreement = (baseline.predicted_class == robust.predicted_class).

8. FRONTEND RESULTS RENDERING:
   Frontend receives PredictResponse JSON. Renders original vs shifted images,
   displays 9-class probability bar charts, highlights prediction flips, and updates BatchResultsTable.
```

7.2 Workflow B: Results Dashboard (/dashboard/)

The Results Dashboard presents the verified Stage B benchmark across the 1,077 held-out test patches:

• **Static Synchronous Ingestion:** On server-side render, `loadBenchmarkDataSync()` parses `public/data/stainshift_results.json` with runtime assertion guards (verifying N=1,077, 8 conditions).
• **Visual Comparison Chart:** `AccuracyComparisonChart.tsx` projects baseline vs robust bars into responsive SVG coordinates using clean short labels (`Hvy. Hema.`, `Deconv.`) to eliminate label cutoff.
• **Interactive Confusion Matrix:** `ConfusionMatrixViewer.tsx` provides a toggle between Baseline and Stain-Robust models, dynamically highlighting true diagonal classifications versus off-diagonal misclassifications.
• **Technical Specs Drawer:** `TechnicalSpecsDrawer.tsx` stores cryptographic verification data (Git Commit, MLflow Run ID, SHA-256 checkpoint checksums) in a collapsible drawer to eliminate visual clutter.

7.3 Workflow C: Precomputed Demo Explorer (/demo/)

The Demo Explorer allows evaluators to inspect 72 representative tissue patches (8 patches per class across all 9 classes) pre-evaluated across all eight stain conditions. Because all images and model predictions are pre-rendered into static JSON manifests (`public/demo/demo_manifest.json`), the Demo Explorer runs with **zero backend connection**, providing a foolproof, zero-latency fallback during live stage demos.

8. Verbatim Source Code References & Module Walkthroughs

8.1 API Request Dispatcher: api/main.py

The FastAPI `predict()` handler accepts multipart uploads, validates boundary limits, and aggregates batch predictions:

FILE: `api/main.py · Request Dispatcher`

```
@app.post('/predict', response_model=PredictResponse)
async def predict(
    files: List[UploadFile] = File(..., description='One or more images or ZIP archive'),
    model_id: str = Form('robust_stain_aug', description="'baseline', 'robust_stain_aug', or 'compare'"),
    condition: str = Form('original', description='Stain shift condition'),
    intensity: float = Form(1.0, description='Perturbation scaling (0.0 to 2.5)'),
) -> PredictResponse:
    if model_id not in ('baseline', 'robust_stain_aug', 'compare', 'both'):
        raise HTTPException(status_code=400, detail=f"Invalid model_id '{model_id}'")
    if condition not in DISTINCT_STAIN_PROFILES:
        raise HTTPException(status_code=400, detail=f"Invalid condition '{condition}'")

    items_to_process = []
    for uf in files:
        content = await uf.read()
        if Path(uf.filename).suffix.lower() == '.zip':
            items_to_process.extend(engine.unpack_zip_archive(content))
        elif Path(uf.filename).suffix.lower() in SUPPORTED_IMAGE_EXTENSIONS:
            items_to_process.append((uf.filename, content))

    results = [ImageInferenceResult(**engine.process_image(
        image_bytes=raw, filename=fname, model_mode=model_id, condition=condition, intensity=intensity
    )) for fname, raw in items_to_process]

    return PredictResponse(status='success', summary=engine.summarize(results), results=results)
```

8.2 Inference Engine: api/inference.py

The `run_single_model_inference()` method handles hardware synchronization and tensor evaluation:

FILE: `api/inference.py · Hardware-Synchronized Inference`

```
def run_single_model_inference(self, model_id: str, tensor_224: torch.Tensor) -> Dict[str, Any]:
    model = self.models[model_id]
    tensor_device = tensor_224.to(self.device)

    # Synchronize compute device before timing
    if self.device_str == 'mps': torch.mps.synchronize()
    elif self.device_str == 'cuda': torch.cuda.synchronize()

    t0 = time.perf_counter()
    with torch.no_grad():
        logits = model(tensor_device)
        probs = F.softmax(logits, dim=1)[0]

    if self.device_str == 'mps': torch.mps.synchronize()
    elif self.device_str == 'cuda': torch.cuda.synchronize()
    t1 = time.perf_counter()

    probs_np = probs.cpu().numpy()
    pred_idx = int(np.argmax(probs_np))
    return {
        'model_id': model_id,
        'predicted_class': CLASS_NAMES[pred_idx],
        'confidence': round(float(probs_np[pred_idx]), 4),
        'probabilities': {c: round(float(probs_np[i]), 4) for i, c in enumerate(CLASS_NAMES)},
        'inference_time_ms': round((t1 - t0) * 1000.0, 3)
    }
```

8.3 Physics Engine: src/stain_shift.py

The Beer-Lambert perturbation logic showing concentration scaling and optical rotation:

FILE: `src/stain_shift.py · Optical Density Perturbation`

```
def perturb(self, img_rgb: np.ndarray, alpha_h: float=1.0, alpha_e: float=1.0, stain_rot_deg: float=0.0):
    # Exact identity bypass for neutral controls
    if abs(alpha_h - 1.0) < 1e-6 and abs(alpha_e - 1.0) < 1e-6 and abs(stain_rot_deg) < 1e-4:
        return img_rgb.copy()

    stain_matrix, concentrations, (h, w) = self.extract_stain_vectors_and_concentrations(img_rgb)
    c_perturbed = concentrations.copy()
    c_perturbed[:, 0] *= alpha_h   # Scale Hematoxylin (nuclear absorption)
    c_perturbed[:, 1] *= alpha_e   # Scale Eosin (cytoplasmic absorption)

    s_perturbed = stain_matrix.copy()
    if abs(stain_rot_deg) > 1e-3:
        rad = np.radians(stain_rot_deg)
        rot_matrix = np.array([
            [np.cos(rad), -np.sin(rad), 0.0],
            [np.sin(rad),  np.cos(rad), 0.0],
            [0.0,          0.0,         1.0]
        ])
        s_perturbed = np.maximum(np.dot(s_perturbed, rot_matrix), 0.0)
        s_perturbed /= np.linalg.norm(s_perturbed, axis=1, keepdims=True)

    od_perturbed = np.dot(c_perturbed, s_perturbed).reshape((h, w, 3))
    return self.od_to_rgb(od_perturbed)  # I = 255.0 * 10^(-OD)
```

9. Developer Learning Guide: Building StainShift from Scratch

Educational Note for Bioinformatics Students:
As a bioinformatics engineer, you are uniquely positioned at the intersection of biological domain knowledge (cellular pathology, tissue histology, dye chemistry) and computational engineering (computer vision, deep neural networks, asynchronous web architectures). Building a production system like StainShift involves ten systematic phases:

Phase 1: Environment Setup & Isolation
Create an isolated Python virtual environment (`python -m venv .venv`). Install pinned dependencies from `requirements.txt`. Never install packages globally. Set up Node.js 18+ and npm for the frontend.

Phase 2: Biological Data Modeling & Splits
Download the CRC-VAL-HE-7K dataset. Histopathology patches must be grouped by slide or patient ID where available. Enforce strict sample disjointness: patches from the same patient/slide must never be split between training and test sets. Sort file paths deterministically before shuffling with a fixed random seed (`seed=42`).

Phase 3: The Physics of Stain Deconvolution
Implement Beer-Lambert optical density deconvolution in pure NumPy. Do not rely on black-box libraries until you understand the math: convert RGB to OD using `-log10(I / 255.0)`, compute the sample covariance matrix, perform PCA via `np.linalg.eigh`, and solve for dye concentrations with least-squares.

Phase 4: PyTorch Model Architecture & Transfer Learning
Load a pretrained MobileNetV3-Small backbone from `torchvision.models`. Replace the final classifier with a linear bottleneck layer (256 units), Hardswish activation, dropout (0.2), and an output projection to 9 logits matching your tissue classes.

Phase 5: The Augmented Training Loop
Train the baseline model with standard geometric transforms (random flips and 90-degree rotations). Train the robust model with online Macenko stain perturbation, randomly scaling α_H, α_E in [0.7, 1.4] and rotating stain planes by ±8°. Track validation F1 scores at every epoch using MLflow.

Phase 6: Multi-Condition Benchmarking Pipeline
Write an evaluation script (`src/evaluate.py`) that evaluates both models across all eight synthetic stain profiles. Export 9x9 confusion matrices, per-class F1 metrics, and save verified checkpoints with SHA-256 hashes.

Phase 7: Building the FastAPI REST Microservice
Construct asynchronous endpoints using FastAPI. Implement Pydantic data contracts for request and response serialization. Implement strict security boundaries: in-memory `io.BytesIO` image parsing, maximum file size limits, and Zip-Bomb protection.

Phase 8: Building the Next.js Frontend UI
Initialize a Next.js 14 project with TypeScript and Tailwind CSS. Design a scrollytelling narrative landing page on the homepage (`/`) and an interactive workspace (`/workspace/`). Use native `IntersectionObserver` and `requestAnimationFrame` for butter-smooth 60fps animations.

Phase 9: Connecting Frontend & Backend via REST
Use the browser's native `fetch` API to send `FormData` to `/predict`. Render original vs transformed patches side-by-side using base64 data URLs. Dynamically highlight prediction flips.

Phase 10: Production Deployment & Edge Distribution
Deploy the static Next.js frontend to Vercel (`output: 'export'`) for zero-latency global edge delivery. Deploy the containerized FastAPI Python backend to Render using Docker. Configure CORS and environment variables.

9.2 Comprehensive Technical Glossary

Optical Density (OD)	A logarithmic measure of light absorption: OD = -log10(I / I_0). In contrast to non-linear RGB values, OD values are strictly linear and additive with respect to dye concentration.
Beer-Lambert Law	The physical equation governing light attenuation through absorbing media: I = I_0 * 10^(-S^T * C). Forms the mathematical foundation for separating Hematoxylin and Eosin dye components.
Macenko Deconvolution	An unsupervised, PCA-based mathematical technique that estimates individual stain absorption vectors directly from an RGB image by finding extreme angular percentiles in the 2D optical density plane.
Reinhard Normalization	A statistical color transfer method that aligns the mean and standard deviation of an image's channels to a reference target in Ruderman l-alpha-beta color space.
Empirical Risk Minimization (ERM)	The standard machine learning paradigm of minimizing average training loss over an empirical sample set. Frequently leads models to memorize spurious color shortcuts.
Domain Shift	A statistical discrepancy between training and deployment data distributions. In digital pathology, stain variation across laboratories is the primary cause of domain shift failure.
Prediction Flip	An event where an identical histological tissue patch is classified into class A under standard conditions, but flips into class B when stain parameters shift.
Softmax Function	A normalized exponential mathematical function that converts unconstrained neural network logits into a probability distribution summing to 1.0.
Macro F1-Score	The unweighted mean of F1-scores across all classes: (1/C) * sum(F1_c). Treats all tissue classes equally regardless of sample imbalance.
Metal Performance Shaders (MPS)	Apple's hardware-accelerated GPU computing framework for Apple Silicon processors, providing accelerated tensor operations via PyTorch's torch.backends.mps backend.
Asynchronous Server Gateway Interface (ASGI)	The modern Python specification for asynchronous web servers, enabling concurrent request processing and non-blocking I/O in FastAPI.
Static Export (SSG)	A Next.js build compilation mode (output: 'export') that pre-renders all pages into static HTML, CSS, and JS bundles, enabling instant CDN delivery with zero server runtime overhead.

10. Verification Sign-Off & Document Integrity

DOCUMENT INTEGRITY & METRIC ATTESTATION:
This technical architecture and developer guide was generated directly from the working source code of the StainShift repository at commit **b890655** on **October 30, 2026**. All software versions, mathematical formulations, code listings, and performance benchmarks were verified through automated empirical regression testing against the deployed Vercel frontend (https://stainshift-ai.vercel.app/) and Render FastAPI backend (https://stainshift-api.onrender.com).

Verified Cryptographic Checkpoint Signatures:
• Baseline Model Checkpoint (`baseline.pth`):
`3b656cb19974a00e825a73590b2fded9d19745cb9c9e0f68ef2af2288735bf76`
• Robust Model Checkpoint (`robust_stain_aug.pth`):
`48ae96612d585c696179e355bcb5d8b12b06dcb651dd9c4ad457b68e446d792a`
• Frozen Dataset Split Manifest (`splits.json`):
`15ad014637fa207d79bd2b02b3adda6bf9ca22035d36cf9f296d04f6bdec0050`

Academic Citations:
[1] Macenko, M. et al. (2009). *A method for normalizing histology slides for quantitative analysis.* IEEE ISBI, pp. 1107-1110.
[2] Kather, J. N. et al. (2019). *Predicting survival from colorectal cancer histology slides using deep learning: A multicenter study.* PLOS Medicine, 16(1): e1002730.
[3] Howard, A. et al. (2019). *Searching for MobileNetV3.* IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1314-1324.
[4] Reinhard, E. et al. (2001). *Color transfer between images.* IEEE Computer Graphics and Applications, 21(5): 34-41.

DOCUMENT CLASSIFICATION: Unrestricted Technical Documentation	**STATUS: STAGE B VERIFIED**
DISTRIBUTION: GitHub Repository & Vercel Public Edge	**COMPLIANCE:** Non-Clinical Investigational Benchmark